Strategy Pattern (Policy-Based Design)

Theory, Edge Cases & Code Examples

- ✓ Strategy Pattern Overview
- ✓ Runtime vs Compile-Time Strategy
- ✓ C++ Implementation Patterns
- ✓ Autonomous Vehicle Example
- ✓ Strategy Pattern Benefits and Trade-offs

→ Swipe to learn →

Strategy Pattern (Policy-Based Design)

Theory

📌 1. Strategy Pattern Overview (1/2)

Definition: Behavioral design pattern defining family of algorithms, encapsulating each one, and making them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

Core Components:

Component	Role	Responsibility
Strategy	Algorithm interface	Defines common interface for all algorithms
Concrete Strategy	Specific algorithm	Implements particular algorithm variant
Context	Algorithm user	Maintains reference to Strategy, delegates work
Client	Strategy selector	Creates context with chosen strategy

Core Guarantee:

Requirement	Implementation	Purpose
Interchangeability	All strategies implement same interface	Swap algorithms at runtime
Encapsulation	Algorithm logic isolated in strategy classes	Hide implementation details
Open/Closed	Add new strategies without modifying context	Extensibility
Composition	Context delegates to strategy	Prefer composition over inheritance

Strategy Pattern (Policy-Based Design)

Theory

📌 1. Strategy Pattern Overview (2/2)

When to Use Strategy Pattern:

Common Use Cases:

```
class PathPlanner {           // ✅ Multiple path planning algorithms (A*, RRT, Dijkstra)
class SortAlgorithm {          // ✅ Different sorting strategies (QuickSort, MergeSort, HeapSort)
class CompressionStrategy {    // ✅ Various compression algorithms (ZIP, GZIP, LZ4)
class PaymentProcessor {       // ✅ Different payment methods (Credit, PayPal, Bitcoin)
class ValidationStrategy {     // ✅ Multiple validation rules (Email, Phone, SSN)
```

Inappropriate Uses:

```
class ConstantAlgorithm {      // ❌ Only one algorithm - no need for strategy
class TightlyCoupledLogic {    // ❌ Algorithm needs context internals - violates encapsulation
class SimpleIfElse {           // ❌ 2-3 simple cases - overkill for strategy pattern
```

Strategy Pattern (Policy-Based Design)

Theory

📌 2. Runtime vs Compile-Time Strategy (1/4)

Runtime Strategy (Traditional - Virtual Functions):

```
// Strategy interface
class PathPlanningStrategy {
public:
    virtual ~PathPlanningStrategy() = default;
    virtual Path computePath(const Point& start, const Point& goal) = 0;
};

// Concrete strategies
class AStarPlanner : public PathPlanningStrategy {
public:
    Path computePath(const Point& start, const Point& goal) override {
        // A* algorithm implementation
        return aStarSearch(start, goal);
    }
};

class RRTPlanner : public PathPlanningStrategy {
public:
    Path computePath(const Point& start, const Point& goal) override {
        // RRT algorithm implementation
        return rrtSearch(start, goal);
    }
};

// Context uses strategy
class Navigator {
    std::unique_ptr<PathPlanningStrategy> strategy;

public:
    Navigator(std::unique_ptr<PathPlanningStrategy> s) : strategy(std::move(s)) {}

    void setStrategy(std::unique_ptr<PathPlanningStrategy> s) {
        strategy = std::move(s);
    }

    Path navigate(const Point& start, const Point& goal) {
        return strategy->computePath(start, goal);  // Delegate to strategy
    }
};
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Theory

📌 2. Runtime vs Compile-Time Strategy (2/4)

[Code continues across slides]

```
// ...continued from previous slide

// Usage - runtime selection
Navigator nav(std::make_unique<AStarPlanner>());
Path path1 = nav.navigate(start, goal);

// Switch strategy at runtime
nav.setStrategy(std::make_unique<RRTPlanner>());
Path path2 = nav.navigate(start, goal);
```

Strategy Pattern (Policy-Based Design)

Theory

📌 2. Runtime vs Compile-Time Strategy (3/4)

Compile-Time Strategy (Policy-Based Design - Templates):

```
// Strategy as template parameter (policy)
template<typename SortPolicy>
class Sorter {
    SortPolicy policy;

public:
    template<typename Iterator>
    void sort(Iterator begin, Iterator end) {
        policy.sort(begin, end);  // Delegate to policy
    }
};

// Policies (strategies)
struct QuickSortPolicy {
    template<typename Iterator>
    void sort(Iterator begin, Iterator end) {
        // QuickSort implementation
        std::sort(begin, end);  // Using std::sort (typically QuickSort)
    }
};

struct MergeSortPolicy {
    template<typename Iterator>
    void sort(Iterator begin, Iterator end) {
        // MergeSort implementation
        std::stable_sort(begin, end);
    }
};

// Usage - compile-time selection
Sorter<QuickSortPolicy> quickSorter;
quickSorter.sort(vec.begin(), vec.end());

Sorter<MergeSortPolicy> mergeSorter;
mergeSorter.sort(vec.begin(), vec.end());
```

Strategy Pattern (Policy-Based Design)

Theory

📌 2. Runtime vs Compile-Time Strategy (4/4)

Comparison:

Aspect	Runtime (Virtual)	Compile-Time (Template)
Performance	~3-5ns overhead (vtable)	Zero overhead (inlined)
Flexibility	Change strategy at runtime	Fixed at compile-time
Binary size	Smaller (one implementation)	Larger (code duplication per type)
Type safety	Runtime polymorphism	Compile-time type checking
Use case	Strategy from config/user input	Known at compile-time, performance critical

Strategy Pattern (Policy-Based Design)

Theory

📌 3. C++ Implementation Patterns (1/4)

A. Function Pointer Strategy (Simple C-style):

```
using PathPlannerFunc = Path (*)(const Point&, const Point&);

class Navigator {
    PathPlannerFunc strategy;

public:
    Navigator(PathPlannerFunc s) : strategy(s) {}

    Path navigate(const Point& start, const Point& goal) {
        return strategy(start, goal);
    }
};

// Functions as strategies
Path aStarPlanning(const Point& start, const Point& goal) { /* ... */ }
Path rrtPlanning(const Point& start, const Point& goal) { /* ... */ }

// Usage
Navigator nav(aStarPlanning);
```

Strategy Pattern (Policy-Based Design)

Theory

📌 3. C++ Implementation Patterns (2/4)

B. std::function Strategy (Flexible):

```
class Validator {
    std::function<bool(const std::string&)> strategy;

public:
    Validator(std::function<bool(const std::string&)> s) : strategy(s) {}

    bool validate(const std::string& input) {
        return strategy(input);
    }
};

// Lambda strategies
Validator emailValidator([](const std::string& s) {
    return s.find('@') != std::string::npos;
});

Validator phoneValidator([](const std::string& s) {
    return s.length() == 10 && std::all_of(s.begin(), s.end(), ::isdigit);
});
```

Strategy Pattern (Policy-Based Design)

Theory

📌 3. C++ Implementation Patterns (3/4)

C. Functor Strategy (Stateful):

```
// Strategy as functor (function object with state)
class ExponentialRetryStrategy {
    int maxRetries;
    int baseDelay;

public:
    ExponentialRetryStrategy(int retries, int delay)
        : maxRetries(retries), baseDelay(delay) {}

    int getDelay(int attempt) const {
        return baseDelay * (1 << attempt);  // 2^attempt exponential backoff
    }

    bool shouldRetry(int attempt) const {
        return attempt < maxRetries;
    }
};

class LinearRetryStrategy {
    int maxRetries;
    int fixedDelay;

public:
    LinearRetryStrategy(int retries, int delay)
        : maxRetries(retries), fixedDelay(delay) {}

    int getDelay(int attempt) const {
        return fixedDelay;  // Fixed delay
    }

    bool shouldRetry(int attempt) const {
        return attempt < maxRetries;
    }
};

template<typename RetryStrategy>
class Connection {
    RetryStrategy retryStrategy;

// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Theory

📌 3. C++ Implementation Patterns (4/4)

[Code continues across slides]

```
// ...continued from previous slide
public:
    Connection(RetryStrategy rs) : retryStrategy(rs) {}

    bool connect() {
        int attempt = 0;
        while (true) {
            if (tryConnect()) return true;

            if (!retryStrategy.shouldRetry(attempt)) {
                return false;  // Max retries exceeded
            }

            int delay = retryStrategy.getDelay(attempt);
            std::this_thread::sleep_for(std::chrono::milliseconds(delay));
            attempt++;
        }
    }
};
```

D. Variant Strategy (Type-Safe Union):

```
using CompressionStrategy = std::variant<ZipCompression, GzipCompression, LZ4Compression>;

class FileCompressor {
    CompressionStrategy strategy;

public:
    FileCompressor(CompressionStrategy s) : strategy(s) {}

    void compress(const std::string& file) {
        std::visit([&file](auto& compressor) {
            compressor.compress(file);  // Calls correct compress method
        }, strategy);
    }
};
```

Strategy Pattern (Policy-Based Design)

Theory

📌 4. Autonomous Vehicle Example (1/3)

Real-World Strategy Use Case:

```
// Strategy interface for path planning
class PathPlanningStrategy {
public:
    virtual ~PathPlanningStrategy() = default;
    virtual Path computePath(const Map& map, const Point& start, const Point& goal) = 0;
    virtual std::string getName() const = 0;
};

// Concrete strategies for different scenarios
class HighwayPlanner : public PathPlanningStrategy {
public:
    Path computePath(const Map& map, const Point& start, const Point& goal) override {
        // Optimized for highway driving: high speed, lane keeping
        std::cout << "Computing highway path (prioritize speed)\n";
        return computeHighSpeedPath(map, start, goal);
    }

    std::string getName() const override { return "Highway Planner"; }
};

class UrbanPlanner : public PathPlanningStrategy {
public:
    Path computePath(const Map& map, const Point& start, const Point& goal) override {
        // Optimized for city driving: pedestrians, traffic lights, complex intersections
        std::cout << "Computing urban path (prioritize safety, handle traffic)\n";
        return computeSafeCityPath(map, start, goal);
    }

    std::string getName() const override { return "Urban Planner"; }
};

class ParkingPlanner : public PathPlanningStrategy {
public:
    Path computePath(const Map& map, const Point& start, const Point& goal) override {
        // Optimized for parking: tight maneuvers, low speed, precise control
        std::cout << "Computing parking path (precise maneuvers)\n";
        return computeParkingManeuver(map, start, goal);
    }

// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Theory

📌 4. Autonomous Vehicle Example (2/3)

```
// ...continued from previous slide
    std::string getName() const override { return "Parking Planner"; }
};

class EmergencyPlanner : public PathPlanningStrategy {
public:
    Path computePath(const Map& map, const Point& start, const Point& goal) override {
        // Emergency evasion: rapid path changes, collision avoidance
        std::cout << "Computing emergency evasive path (collision avoidance)\n";
        return computeEmergencyEvasion(map, start, goal);
    }

    std::string getName() const override { return "Emergency Planner"; }
};

// Context: Autonomous vehicle path planner
class VehiclePathPlanner {
    std::unique_ptr<PathPlanningStrategy> currentStrategy;

public:
    VehiclePathPlanner(std::unique_ptr<PathPlanningStrategy> strategy)
        : currentStrategy(std::move(strategy)) {}

    void selectStrategy(std::unique_ptr<PathPlanningStrategy> strategy) {
        std::cout << "Switching from " << currentStrategy->getName()
                  << " to " << strategy->getName() << "\n";
        currentStrategy = std::move(strategy);
    }

    Path planPath(const Map& map, const Point& start, const Point& goal) {
        return currentStrategy->computePath(map, start, goal);
    }
};

// Usage: Vehicle switches strategies based on driving scenario
VehiclePathPlanner planner(std::make_unique<HighwayPlanner>());
planner.planPath(map, currentPos, destination);  // Highway planning

// Entering city - switch strategy
planner.selectStrategy(std::make_unique<UrbanPlanner>());
planner.planPath(map, currentPos, destination);  // Urban planning
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Theory

📌 4. Autonomous Vehicle Example (3/3)

[Code continues across slides]

```
// ...continued from previous slide

// Emergency obstacle detected - switch strategy
planner.selectStrategy(std::make_unique<EmergencyPlanner>());
planner.planPath(map, currentPos, safeLocation);  // Emergency evasion
```

Strategy Pattern (Policy-Based Design)

Theory

📌 5. Strategy Pattern Benefits and Trade-offs

Benefits:

Benefit	Explanation	Example
Runtime flexibility	Switch algorithms at runtime	Change path planner based on road type
Open/Closed Principle	Add new strategies without modifying context	Add new planning algorithm without changing Navigator
Eliminate conditionals	Replace if/switch with polymorphism	No if(highway) vs if(urban) chains
Testability	Test each strategy independently	Unit test each planner in isolation
Reusability	Strategies reusable in different contexts	Same sorting strategy in different containers

Trade-offs:

Drawback	Explanation	Mitigation
Overhead	Virtual call indirection (~3-5ns)	Use compile-time strategy (templates) for performance
Object proliferation	Many strategy classes	Use lambdas or std::function for simple strategies
Context coupling	Strategies need context data	Pass data as parameters or provide context interface
Client complexity	Client must know all strategies	Use factory or config to select strategy

Strategy Pattern (Policy-Based Design)

Theory

📌 6. Why Strategy Pattern Matters

Critical Concepts Demonstrated:

Concept	How Strategy Tests It	Interview Relevance
Polymorphism	Strategies accessed through interface	Core OOP understanding
Composition	Context has-a strategy (not is-a)	Prefer composition over inheritance
Open/Closed	Extend with new strategies, don't modify	SOLID principles
Single Responsibility	Each strategy has one algorithm	Separation of concerns
Dependency Inversion	Depend on abstraction, not concrete strategy	Decoupling

Common Interview Questions:

- "What's the difference between Strategy and State pattern?" (Strategy: client chooses; State: self-transitions)
- "Strategy vs Template Method pattern?" (Strategy: composition; Template Method: inheritance)
- "When to use runtime vs compile-time strategy?" (Runtime: flexibility; Compile-time: performance)
- "How does Strategy eliminate conditionals?" (Polymorphism replaces if/switch)

Strategy Pattern (Policy-Based Design)

Edge Cases

📌 1. Strategy Needing Context Data (Circular Dependency) (1/2)

```
// ❌ PROBLEM: Strategy needs context data, creates circular dependency
class Navigator {
    std::unique_ptr<PathPlanningStrategy> strategy;
    Map map;  // Context has map

public:
    Path navigate(const Point& start, const Point& goal) {
        // ❌ Strategy needs map, but passing 'this' creates coupling
        return strategy->computePath(this, start, goal);
    }
};

class PathPlanningStrategy {
public:
    virtual Path computePath(Navigator* context, const Point& start, const Point& goal) = 0;
    // ❌ Strategy now depends on Navigator - circular dependency
};

// ✅ SOLUTION 1: Pass data as parameters
class PathPlanningStrategy {
public:
    virtual Path computePath(const Map& map, const Point& start, const Point& goal) = 0;
};

class Navigator {
public:
    Path navigate(const Point& start, const Point& goal) {
        return strategy->computePath(map, start, goal);  // ✅ Pass needed data
    }
};

// ✅ SOLUTION 2: Context interface (minimal coupling)
class INavigatorContext {
public:
    virtual const Map& getMap() const = 0;
    virtual double getMaxSpeed() const = 0;
};

class Navigator : public INavigatorContext {
    // Implement interface
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Edge Cases

📌 1. Strategy Needing Context Data (Circular Dependency) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
};

class PathPlanningStrategy {
public:
    virtual Path computePath(INavigatorContext& context, const Point& start, const Point& goal) = 0;
    // ✅ Depends only on interface, not concrete Navigator
};
```

Why This Matters: Strategies often need context data (maps, configuration, state). Passing entire context creates tight coupling. Pass only needed data as parameters or define minimal context interface.

Strategy Pattern (Policy-Based Design)

Edge Cases

📌 2. Stateful Strategies and Thread Safety (1/2)

```
// ❌ PROBLEM: Stateful strategy used concurrently is not thread-safe
class CachingStrategy {
    std::map<std::string, int> cache;  // ❌ Mutable state

public:
    int compute(const std::string& key) {
        auto it = cache.find(key);
        if (it != cache.end()) {
            return it->second;  // Cache hit
        }

        int result = expensiveComputation(key);
        cache[key] = result;  // ❌ Data race if multiple threads access
        return result;
    }
};

// Multiple threads use same strategy instance
Navigator nav(std::make_shared<CachingStrategy>());

// Thread 1: nav.navigate(...)
// Thread 2: nav.navigate(...)  // ❌ Data race on cache!

// ✅ SOLUTION 1: Thread-safe strategy with mutex
class ThreadSafeCachingStrategy {
    mutable std::mutex mtx;
    std::map<std::string, int> cache;

public:
    int compute(const std::string& key) {
        std::lock_guard lock(mtx);
        // ... same logic, now thread-safe
    }
};

// ✅ SOLUTION 2: Stateless strategy
class StatelessStrategy {
public:
    int compute(const std::string& key) {
        return expensiveComputation(key);  // No state, thread-safe
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Edge Cases

📌 2. Stateful Strategies and Thread Safety (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }
};

// ✅ SOLUTION 3: Thread-local strategy instances
thread_local CachingStrategy localStrategy;  // Each thread has own instance
```

Why This Matters: Strategies with state (cache, counters, buffers) must be thread-safe if shared. Prefer stateless strategies or use thread-local storage for per-thread state.

Strategy Pattern (Policy-Based Design)

Edge Cases

📌 3. Strategy Lifetime and Ownership (1/2)

```
// ❌ PROBLEM: Strategy lifetime shorter than context
void setupNavigator(Navigator& nav) {
    AStarPlanner planner;
    nav.setStrategy(&planner);  // ❌ Dangling pointer when planner destroyed
}  // planner destroyed here

nav.navigate(...);  // ❌ Use-after-free!

// ✅ SOLUTION 1: Context owns strategy (unique_ptr)
class Navigator {
    std::unique_ptr<PathPlanningStrategy> strategy;

public:
    void setStrategy(std::unique_ptr<PathPlanningStrategy> s) {
        strategy = std::move(s);  // ✅ Takes ownership
    }
};

nav.setStrategy(std::make_unique<AStarPlanner>());  // ✅ Ownership transferred

// ✅ SOLUTION 2: Shared ownership (shared_ptr)
class Navigator {
    std::shared_ptr<PathPlanningStrategy> strategy;

public:
    void setStrategy(std::shared_ptr<PathPlanningStrategy> s) {
        strategy = s;  // ✅ Shared ownership
    }
};

auto planner = std::make_shared<AStarPlanner>();
nav1.setStrategy(planner);
nav2.setStrategy(planner);  // Both share same strategy

// ✅ SOLUTION 3: Strategy stored by value (if small)
class Navigator {
    PathPlanningStrategy strategy;  // ❌ Can't use polymorphism with value

    // Alternative: std::variant for known strategies
    std::variant<AStarPlanner, RRTPlanner> strategy;
};
```

Strategy Pattern (Policy-Based Design)

Edge Cases

📌 3. Strategy Lifetime and Ownership (2/2)

Why This Matters: Strategy lifetime must exceed context lifetime. Use smart pointers (unique_ptr for exclusive, shared_ptr for shared) to manage ownership. Avoid raw pointers to prevent dangling references.

Strategy Pattern (Policy-Based Design)

Edge Cases

📌 4. Default Strategy and Null Strategy Pattern (1/2)

```
// ❌ PROBLEM: No strategy set, nullptr dereference
class Navigator {
    std::unique_ptr<PathPlanningStrategy> strategy;  // May be null

public:
    Path navigate(const Point& start, const Point& goal) {
        return strategy->computePath(...);  // ❌ Crash if strategy is null!
    }
};

Navigator nav;  // No strategy set
nav.navigate(...);  // ❌ Nullptr dereference!

// ✅ SOLUTION 1: Require strategy in constructor
class Navigator {
    std::unique_ptr<PathPlanningStrategy> strategy;

public:
    Navigator(std::unique_ptr<PathPlanningStrategy> s) : strategy(std::move(s)) {
        if (!strategy) throw std::invalid_argument("Strategy required");
    }
    // ✅ Always has valid strategy
};

// ✅ SOLUTION 2: Default strategy
class DefaultPathPlanner : public PathPlanningStrategy {
public:
    Path computePath(...) override {
        return computeSimplePath(...);  // Basic fallback algorithm
    }
};

class Navigator {
    std::unique_ptr<PathPlanningStrategy> strategy;

public:
    Navigator() : strategy(std::make_unique<DefaultPathPlanner>()) {
        // ✅ Always has strategy (default if not set)
    }
};
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Edge Cases

📌 4. Default Strategy and Null Strategy Pattern (2/2)

[Code continues across slides]

```
// ...continued from previous slide

// ✅ SOLUTION 3: Null Object Pattern
class NullPathPlanner : public PathPlanningStrategy {
public:
    Path computePath(...) override {
        return Path{};  // Empty path (do-nothing strategy)
    }
};

class Navigator {
    std::unique_ptr<PathPlanningStrategy> strategy =
        std::make_unique<NullPathPlanner>();  // ✅ Never null
};
```

Why This Matters: Context without strategy causes nullptr dereference. Require strategy in constructor, provide default strategy, or use Null Object pattern (do-nothing strategy).

Strategy Pattern (Policy-Based Design)

Edge Cases

📌 5. Strategy Selection Logic Complexity (1/2)

```
// ❌ PROBLEM: Complex strategy selection logic scattered in client code
void processFile(const std::string& filename) {
    std::unique_ptr<CompressionStrategy> strategy;

    if (filename.ends_with(".txt") || filename.ends_with(".log")) {
        if (fileSize(filename) > 10'000'000) {
            strategy = std::make_unique<LZ4Compression>();  // Fast for large files
        } else {
            strategy = std::make_unique<GzipCompression>();  // Better ratio for small
        }
    } else if (filename.ends_with(".json") || filename.ends_with(".xml")) {
        strategy = std::make_unique<ZipCompression>();
    } else {
        strategy = std::make_unique<NoCompression>();
    }

    Compressor compressor(std::move(strategy));
    compressor.compress(filename);
}

// ✅ SOLUTION: Strategy Factory encapsulates selection logic
class CompressionStrategyFactory {
public:
    static std::unique_ptr<CompressionStrategy> createStrategy(
        const std::string& filename
    ) {
        if (filename.ends_with(".txt") || filename.ends_with(".log")) {
            if (fileSize(filename) > 10'000'000) {
                return std::make_unique<LZ4Compression>();
            }
            return std::make_unique<GzipCompression>();
        } else if (filename.ends_with(".json") || filename.ends_with(".xml")) {
            return std::make_unique<ZipCompression>();
        }
        return std::make_unique<NoCompression>();
    }
};

// Client code simplified
void processFile(const std::string& filename) {
    auto strategy = CompressionStrategyFactory::createStrategy(filename);
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Edge Cases

📌 5. Strategy Selection Logic Complexity (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    Compressor compressor(std::move(strategy));
    compressor.compress(filename);
}
```

Why This Matters: Complex strategy selection logic shouldn't be in client code. Use Factory pattern to encapsulate strategy selection based on conditions (file type, size, configuration).

Strategy Pattern (Policy-Based Design)

Edge Cases

📌 6. Changing Strategy Mid-Execution (1/2)

```
// ❌ PROBLEM: Changing strategy during execution corrupts state
class DataProcessor {
    std::unique_ptr<ProcessingStrategy> strategy;
    std::vector<Data> processedData;  // Accumulated results

public:
    void processItem(const Data& item) {
        Data result = strategy->process(item);
        processedData.push_back(result);
    }

    void changeStrategy(std::unique_ptr<ProcessingStrategy> newStrategy) {
        strategy = std::move(newStrategy);  // ❌ Changes mid-processing!
    }

    std::vector<Data> getResults() { return processedData; }
};

// Usage
processor.changeStrategy(strategyA);
processor.processItem(item1);  // Uses strategyA
processor.changeStrategy(strategyB);  // ❌ Switch mid-processing
processor.processItem(item2);  // Uses strategyB - inconsistent results!

// ✅ SOLUTION 1: Prevent mid-execution changes
class DataProcessor {
    bool processing = false;

public:
    void processItem(const Data& item) {
        processing = true;
        // ... process ...
        processing = false;
    }

    void changeStrategy(std::unique_ptr<ProcessingStrategy> newStrategy) {
        if (processing) {
            throw std::runtime_error("Cannot change strategy during processing");
        }
        strategy = std::move(newStrategy);
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Edge Cases

📌 6. Changing Strategy Mid-Execution (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }
};

// ✅ SOLUTION 2: Complete current batch before changing
class DataProcessor {
public:
    void processBatch(const std::vector<Data>& items) {
        for (const auto& item : items) {
            processItem(item);  // All use same strategy
        }
        // Strategy can change between batches
    }
};

// ✅ SOLUTION 3: Strategy per task (immutable)
class Task {
    std::unique_ptr<ProcessingStrategy> strategy;  // Fixed for task lifetime

public:
    Task(std::unique_ptr<ProcessingStrategy> s) : strategy(std::move(s)) {}
    // Strategy never changes for this task
};
```

Why This Matters: Changing strategy mid-execution can corrupt state or produce inconsistent results. Prevent strategy changes during processing, or ensure strategy is fixed for entire operation.

Strategy Pattern (Policy-Based Design)

Code Examples

📌 1. Easy - Function Pointer Strategy (1/2)

Simple strategy using function pointers. No classes needed for strategies, just free functions. Easy to understand but limited (no state, no polymorphism).

```
#include <iostream>
#include <string>

// Strategy as function pointer
using ValidationFunc = bool (*)(const std::string&);

// Concrete strategies (free functions)
bool validateEmail(const std::string& input) {
    return input.find('@') != std::string::npos &&
           input.find('.') != std::string::npos;
}

bool validatePhone(const std::string& input) {
    if (input.length() != 10) return false;

    for (char c : input) {
        if (!std::isdigit(c)) return false;
    }
    return true;
}

bool validateZipCode(const std::string& input) {
    return input.length() == 5 &&
           std::all_of(input.begin(), input.end(), ::isdigit);
}

// Context
class Validator {
    ValidationFunc strategy;

public:
    Validator(ValidationFunc s) : strategy(s) {}

    void setStrategy(ValidationFunc s) {
        strategy = s;
    }
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 1. Easy - Function Pointer Strategy (2/2)

[Code continues across slides]

```
// ...continued from previous slide

    bool validate(const std::string& input) {
        return strategy(input);
    }
};

int main() {
    std::cout << "Function Pointer Strategy Example\n\n";

    Validator validator(validateEmail);

    std::cout << "Email validation:\n";
    std::cout << "  'user@example.com': "
              << (validator.validate("user@example.com") ? "VALID" : "INVALID") << "\n";
    std::cout << "  'invalid': "
              << (validator.validate("invalid") ? "VALID" : "INVALID") << "\n\n";

    std::cout << "Switching to phone validation:\n";
    validator.setStrategy(validatePhone);

    std::cout << "  '1234567890': "
              << (validator.validate("1234567890") ? "VALID" : "INVALID") << "\n";
    std::cout << "  '123': "
              << (validator.validate("123") ? "VALID" : "INVALID") << "\n";

    return 0;
}
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 2. Mid - Lambda Strategy with std::function (1/2)

Strategy using `std::function` allows lambdas as strategies. More flexible than function pointers - can capture state. Clean syntax for inline strategies.

```
#include <iostream>
#include <functional>
#include <vector>
#include <algorithm>

// Context using std::function
class DataFilter {
    std::function<bool(int)> predicate;

public:
    DataFilter(std::function<bool(int)> p) : predicate(p) {}

    void setPredicate(std::function<bool(int)> p) {
        predicate = p;
    }

    std::vector<int> filter(const std::vector<int>& data) {
        std::vector<int> result;
        for (int val : data) {
            if (predicate(val)) {
                result.push_back(val);
            }
        }
        return result;
    }
};

int main() {
    std::cout << "Lambda Strategy Example\n\n";

    std::vector<int> data = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    // Strategy 1: Filter even numbers
    DataFilter filter([](int x) { return x % 2 == 0; });

    auto evens = filter.filter(data);
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 2. Mid - Lambda Strategy with std::function (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    std::cout << "Even numbers: ";
    for (int x : evens) std::cout << x << " ";
    std::cout << "\n";

    // Strategy 2: Filter numbers greater than 5
    filter.setPredicate([](int x) { return x > 5; });

    auto greaterThan5 = filter.filter(data);
    std::cout << "Greater than 5: ";
    for (int x : greaterThan5) std::cout << x << " ";
    std::cout << "\n";

    // Strategy 3: Filter with captured state
    int threshold = 3;
    filter.setPredicate([threshold](int x) { return x > threshold; });

    auto greaterThanThreshold = filter.filter(data);
    std::cout << "Greater than " << threshold << ": ";
    for (int x : greaterThanThreshold) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 3. Mid - Classic Strategy Pattern (Virtual Functions) (1/4)

Classic Strategy pattern with virtual functions. Three different sorting algorithms encapsulated as strategies. Context (Sorter) delegates to current strategy.

```
#include <iostream>
#include <memory>
#include <vector>

// Strategy interface
class SortStrategy {
public:
    virtual ~SortStrategy() = default;
    virtual void sort(std::vector<int>& data) = 0;
    virtual std::string getName() const = 0;
};

// Concrete strategies
class BubbleSort : public SortStrategy {
public:
    void sort(std::vector<int>& data) override {
        for (size_t i = 0; i < data.size(); ++i) {
            for (size_t j = 0; j < data.size() - i - 1; ++j) {
                if (data[j] > data[j + 1]) {
                    std::swap(data[j], data[j + 1]);
                }
            }
        }
    }

    std::string getName() const override { return "Bubble Sort"; }
};

class QuickSort : public SortStrategy {
    void quicksort(std::vector<int>& data, int low, int high) {
        if (low < high) {
            int pivot = partition(data, low, high);
            quicksort(data, low, pivot - 1);
            quicksort(data, pivot + 1, high);
        }
    }
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 3. Mid - Classic Strategy Pattern (Virtual Functions) (2/4)

```
// ...continued from previous slide

    int partition(std::vector<int>& data, int low, int high) {
        int pivot = data[high];
        int i = low - 1;

        for (int j = low; j < high; ++j) {
            if (data[j] < pivot) {
                ++i;
                std::swap(data[i], data[j]);
            }
        }
        std::swap(data[i + 1], data[high]);
        return i + 1;
    }

public:
    void sort(std::vector<int>& data) override {
        if (!data.empty()) {
            quicksort(data, 0, data.size() - 1);
        }
    }

    std::string getName() const override { return "Quick Sort"; }
};

class MergeSort : public SortStrategy {
    void merge(std::vector<int>& data, int left, int mid, int right) {
        std::vector<int> temp(right - left + 1);
        int i = left, j = mid + 1, k = 0;

        while (i <= mid && j <= right) {
            if (data[i] <= data[j]) {
                temp[k++] = data[i++];
            } else {
                temp[k++] = data[j++];
            }
        }

        while (i <= mid) temp[k++] = data[i++];
        while (j <= right) temp[k++] = data[j++];
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 3. Mid - Classic Strategy Pattern (Virtual Functions) (3/4)

```
// ...continued from previous slide

        for (int i = 0; i < k; ++i) {
            data[left + i] = temp[i];
        }
    }

    void mergesort(std::vector<int>& data, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;
            mergesort(data, left, mid);
            mergesort(data, mid + 1, right);
            merge(data, left, mid, right);
        }
    }

public:
    void sort(std::vector<int>& data) override {
        if (!data.empty()) {
            mergesort(data, 0, data.size() - 1);
        }
    }

    std::string getName() const override { return "Merge Sort"; }
};

// Context
class Sorter {
    std::unique_ptr<SortStrategy> strategy;

public:
    Sorter(std::unique_ptr<SortStrategy> s) : strategy(std::move(s)) {}

    void setStrategy(std::unique_ptr<SortStrategy> s) {
        strategy = std::move(s);
    }

    void performSort(std::vector<int>& data) {
        std::cout << "Using " << strategy->getName() << "\n";
        strategy->sort(data);
    }
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 3. Mid - Classic Strategy Pattern (Virtual Functions) (4/4)

[Code continues across slides]

```
// ...continued from previous slide
};

int main() {
    std::cout << "Classic Strategy Pattern Example\n\n";

    std::vector<int> data1 = {64, 34, 25, 12, 22, 11, 90};
    std::vector<int> data2 = data1;
    std::vector<int> data3 = data1;

    std::cout << "Original data: ";
    for (int x : data1) std::cout << x << " ";
    std::cout << "\n\n";

    Sorter sorter(std::make_unique<BubbleSort>());
    sorter.performSort(data1);
    std::cout << "Result: ";
    for (int x : data1) std::cout << x << " ";
    std::cout << "\n\n";

    sorter.setStrategy(std::make_unique<QuickSort>());
    sorter.performSort(data2);
    std::cout << "Result: ";
    for (int x : data2) std::cout << x << " ";
    std::cout << "\n\n";

    sorter.setStrategy(std::make_unique<MergeSort>());
    sorter.performSort(data3);
    std::cout << "Result: ";
    for (int x : data3) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 4. Advanced - Template-Based Policy (Compile-Time Strategy) (1/2)

Compile-time strategy using templates (policy-based design). Zero runtime overhead - strategies inlined by compiler. Trade-off: less flexibility (fixed at compile-time) for better performance.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <chrono>

// Policy: Sorting algorithms
struct QuickSortPolicy {
    template<typename Iterator>
    static void sort(Iterator begin, Iterator end) {
        std::sort(begin, end);  // Typically QuickSort
    }

    static const char* name() { return "QuickSort Policy"; }
};

struct StableSortPolicy {
    template<typename Iterator>
    static void sort(Iterator begin, Iterator end) {
        std::stable_sort(begin, end);  // MergeSort
    }

    static const char* name() { return "StableSort Policy"; }
};

// Context with compile-time policy
template<typename SortPolicy>
class Sorter {
public:
    template<typename Iterator>
    void performSort(Iterator begin, Iterator end) {
        std::cout << "Using " << SortPolicy::name() << "\n";

        auto start = std::chrono::high_resolution_clock::now();
        SortPolicy::sort(begin, end);
        auto stop = std::chrono::high_resolution_clock::now();

// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 4. Advanced - Template-Based Policy (Compile-Time Strategy) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        auto duration = std::chrono::duration_cast<std::chrono::microseconds>(stop - start);
        std::cout << "Time: " << duration.count() << " µs\n";
    }
};

int main() {
    std::cout << "Template Policy Strategy Example\n\n";

    std::vector<int> data1(10000);
    std::vector<int> data2(10000);

    // Fill with random data
    for (size_t i = 0; i < data1.size(); ++i) {
        data1[i] = rand() % 10000;
        data2[i] = data1[i];
    }

    Sorter<QuickSortPolicy> quickSorter;
    quickSorter.performSort(data1.begin(), data1.end());

    std::cout << "\n";

    Sorter<StableSortPolicy> stableSorter;
    stableSorter.performSort(data2.begin(), data2.end());

    std::cout << "\nNote: Template policies have ZERO runtime overhead!\n";
    std::cout << "Strategies are inlined at compile-time.\n";

    return 0;
}
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 5. Advanced - Strategy with State (Retry Strategies) (1/4)

Strategies with internal state (maxRetries, baseDelay). Shows how stateful strategies encapsulate both algorithm and configuration. Different retry strategies for different failure scenarios.

```
#include <iostream>
#include <memory>
#include <thread>
#include <chrono>

// Strategy interface
class RetryStrategy {
public:
    virtual ~RetryStrategy() = default;
    virtual int getDelay(int attempt) const = 0;
    virtual bool shouldRetry(int attempt) const = 0;
    virtual std::string getName() const = 0;
};

// Concrete strategies with state
class ExponentialBackoff : public RetryStrategy {
    int maxRetries;
    int baseDelay;  // milliseconds

public:
    ExponentialBackoff(int retries, int delay)
        : maxRetries(retries), baseDelay(delay) {}

    int getDelay(int attempt) const override {
        return baseDelay * (1 << attempt);  // 2^attempt exponential growth
    }

    bool shouldRetry(int attempt) const override {
        return attempt < maxRetries;
    }

    std::string getName() const override {
        return "Exponential Backoff";
    }
};

// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 5. Advanced - Strategy with State (Retry Strategies) (2/4)

```
// ...continued from previous slide
class LinearBackoff : public RetryStrategy {
    int maxRetries;
    int fixedDelay;

public:
    LinearBackoff(int retries, int delay)
        : maxRetries(retries), fixedDelay(delay) {}

    int getDelay(int attempt) const override {
        return fixedDelay;  // Fixed delay each time
    }

    bool shouldRetry(int attempt) const override {
        return attempt < maxRetries;
    }

    std::string getName() const override {
        return "Linear Backoff";
    }
};

class FibonacciBackoff : public RetryStrategy {
    int maxRetries;
    int baseDelay;

    int fibonacci(int n) const {
        if (n <= 1) return n;
        int a = 0, b = 1;
        for (int i = 2; i <= n; ++i) {
            int temp = a + b;
            a = b;
            b = temp;
        }
        return b;
    }

public:
    FibonacciBackoff(int retries, int delay)
        : maxRetries(retries), baseDelay(delay) {}

// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 5. Advanced - Strategy with State (Retry Strategies) (3/4)

```
// ...continued from previous slide
    int getDelay(int attempt) const override {
        return baseDelay * fibonacci(attempt);
    }

    bool shouldRetry(int attempt) const override {
        return attempt < maxRetries;
    }

    std::string getName() const override {
        return "Fibonacci Backoff";
    }
};

// Context
class ConnectionManager {
    std::unique_ptr<RetryStrategy> retryStrategy;
    int failureRate;  // 0-100 percentage

public:
    ConnectionManager(std::unique_ptr<RetryStrategy> strategy, int failRate)
        : retryStrategy(std::move(strategy)), failureRate(failRate) {}

    bool connect() {
        std::cout << "Attempting connection with " << retryStrategy->getName() << "\n";

        int attempt = 0;
        while (true) {
            std::cout << "  Attempt " << (attempt + 1) << ": ";

            // Simulate connection (random failure based on failureRate)
            if (rand() % 100 >= failureRate) {
                std::cout << "SUCCESS!\n";
                return true;
            }

            std::cout << "FAILED\n";

            if (!retryStrategy->shouldRetry(attempt)) {
                std::cout << "  Max retries exceeded. Giving up.\n";
                return false;
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 5. Advanced - Strategy with State (Retry Strategies) (4/4)

[Code continues across slides]

```
// ...continued from previous slide
            }

            int delay = retryStrategy->getDelay(attempt);
            std::cout << "  Waiting " << delay << "ms before retry...\n";
            std::this_thread::sleep_for(std::chrono::milliseconds(delay));

            attempt++;
        }
    }
};

int main() {
    std::cout << "Strategy with State Example (Retry Strategies)\n\n";

    std::cout << "=== Exponential Backoff ===\n";
    ConnectionManager conn1(std::make_unique<ExponentialBackoff>(5, 100), 70);
    conn1.connect();

    std::cout << "\n=== Linear Backoff ===\n";
    ConnectionManager conn2(std::make_unique<LinearBackoff>(5, 200), 70);
    conn2.connect();

    std::cout << "\n=== Fibonacci Backoff ===\n";
    ConnectionManager conn3(std::make_unique<FibonacciBackoff>(5, 50), 70);
    conn3.connect();

    return 0;
}
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 7. Advanced - Strategy Factory with Configuration (1/3)

Strategy factory with registry and smart selection. Encapsulates strategy selection logic. Can load strategy from configuration or auto-select based on characteristics.

```
#include <iostream>
#include <memory>
#include <map>
#include <string>
#include <functional>

// Strategy interface
class CompressionStrategy {
public:
    virtual ~CompressionStrategy() = default;
    virtual std::string compress(const std::string& data) = 0;
    virtual std::string getName() const = 0;
};

// Concrete strategies
class ZipCompression : public CompressionStrategy {
public:
    std::string compress(const std::string& data) override {
        return "[ZIP:" + data + "]";  // Simulated compression
    }
    std::string getName() const override { return "ZIP"; }
};

class GzipCompression : public CompressionStrategy {
public:
    std::string compress(const std::string& data) override {
        return "[GZIP:" + data + "]";
    }
    std::string getName() const override { return "GZIP"; }
};

class LZ4Compression : public CompressionStrategy {
public:
    std::string compress(const std::string& data) override {
        return "[LZ4:" + data + "]";
    }
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 7. Advanced - Strategy Factory with Configuration (2/3)

```
// ...continued from previous slide
    std::string getName() const override { return "LZ4"; }
};

// Strategy Factory
class CompressionStrategyFactory {
    using Creator = std::function<std::unique_ptr<CompressionStrategy>()>;
    static std::map<std::string, Creator>& getRegistry() {
        static std::map<std::string, Creator> registry;
        return registry;
    }

public:
    static void registerStrategy(const std::string& name, Creator creator) {
        getRegistry()[name] = creator;
    }

    static std::unique_ptr<CompressionStrategy> create(const std::string& name) {
        auto& registry = getRegistry();
        auto it = registry.find(name);
        if (it == registry.end()) {
            throw std::runtime_error("Unknown compression strategy: " + name);
        }
        return it->second();
    }

    // Smart selection based on file characteristics
    static std::unique_ptr<CompressionStrategy> selectBest(
        size_t fileSize, const std::string& fileType
    ) {
        if (fileSize > 10'000'000) {
            return std::make_unique<LZ4Compression>();  // Fast for large files
        } else if (fileType == "text") {
            return std::make_unique<GzipCompression>();  // Best ratio for text
        } else {
            return std::make_unique<ZipCompression>();  // Default
        }
    }
};

int main() {
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 7. Advanced - Strategy Factory with Configuration (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    std::cout << "Strategy Factory Example\n\n";

    // Register strategies
    CompressionStrategyFactory::registerStrategy("zip", []() {
        return std::make_unique<ZipCompression>();
    });
    CompressionStrategyFactory::registerStrategy("gzip", []() {
        return std::make_unique<GzipCompression>();
    });
    CompressionStrategyFactory::registerStrategy("lz4", []() {
        return std::make_unique<LZ4Compression>();
    });

    // Create from configuration (string)
    auto strategy1 = CompressionStrategyFactory::create("gzip");
    std::cout << "Strategy: " << strategy1->getName() << "\n";
    std::cout << "Result: " << strategy1->compress("Hello World") << "\n\n";

    // Smart selection based on file characteristics
    auto strategy2 = CompressionStrategyFactory::selectBest(15'000'000, "binary");
    std::cout << "Auto-selected for large file: " << strategy2->getName() << "\n";
    std::cout << "Result: " << strategy2->compress("Large data...") << "\n\n";

    auto strategy3 = CompressionStrategyFactory::selectBest(5000, "text");
    std::cout << "Auto-selected for small text: " << strategy3->getName() << "\n";
    std::cout << "Result: " << strategy3->compress("Small text data") << "\n";

    return 0;
}
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 8. Performance - Benchmark Runtime vs Compile-Time Strategy (1/3)

Performance comparison between runtime (virtual) and compile-time (template) strategies. Shows ~2-3x speedup with templates due to inlining. Trade-off: flexibility vs performance.

```
#include <iostream>
#include <vector>
#include <chrono>
#include <memory>

// Runtime strategy (virtual)
class RuntimeStrategy {
public:
    virtual ~RuntimeStrategy() = default;
    virtual int compute(int x) = 0;
};

class RuntimeDouble : public RuntimeStrategy {
public:
    int compute(int x) override { return x * 2; }
};

class RuntimeTriple : public RuntimeStrategy {
public:
    int compute(int x) override { return x * 3; }
};

// Compile-time strategy (template)
struct DoublePolicy {
    static int compute(int x) { return x * 2; }
};

struct TriplePolicy {
    static int compute(int x) { return x * 3; }
};

template<typename Policy>
class CompileTimeStrategy {
public:
    int compute(int x) {
        return Policy::compute(x);
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 8. Performance - Benchmark Runtime vs Compile-Time Strategy (2/3)

```
// ...continued from previous slide
    }
};

int main() {
    std::cout << "Performance: Runtime vs Compile-Time Strategy\n\n";

    const int iterations = 10'000'000;

    // Runtime strategy benchmark
    {
        std::unique_ptr<RuntimeStrategy> strategy = std::make_unique<RuntimeDouble>();
        int sum = 0;

        auto start = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < iterations; ++i) {
            sum += strategy->compute(i);  // Virtual call
        }
        auto stop = std::chrono::high_resolution_clock::now();

        auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(stop - start);
        std::cout << "Runtime Strategy (Virtual):\n";
        std::cout << "  Time: " << duration.count() << " ms\n";
        std::cout << "  Sum: " << sum << " (prevent optimization)\n\n";
    }

    // Compile-time strategy benchmark
    {
        CompileTimeStrategy<DoublePolicy> strategy;
        int sum = 0;

        auto start = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < iterations; ++i) {
            sum += strategy.compute(i);  // Inlined
        }
        auto stop = std::chrono::high_resolution_clock::now();

        auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(stop - start);
        std::cout << "Compile-Time Strategy (Template):\n";
        std::cout << "  Time: " << duration.count() << " ms\n";
        std::cout << "  Sum: " << sum << " (prevent optimization)\n\n";
// ...continues on next slide
```

Strategy Pattern (Policy-Based Design)

Code Examples

📌 8. Performance - Benchmark Runtime vs Compile-Time Strategy (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    }

    std::cout << "Performance Difference:\n";
    std::cout << "  Compile-time strategy is typically 2-3x faster\n";
    std::cout << "  due to inlining and lack of virtual call overhead.\n";

    return 0;
}
```

What You Learned

Key Takeaways

- ✓ Strategy Pattern Overview
- ✓ Runtime vs Compile-Time Strategy
- ✓ C++ Implementation Patterns
- ✓ Autonomous Vehicle Example

→ Swipe to learn →